Enterprise Network Perimeter Security & DoS Attack Mitigation Lab

(pfSense, Kali Linux, Ubuntu, Wireshark)

Mihith Kodagoda
Mihithkodagoda622@gmail.com

1. Project Overview and Objectives

This report documents the setup, testing, and traffic analysis of a virtualized network security lab built in VMware Workstation. The main objective of this project was to build a realistic perimeter firewall environment using pfSense Community Edition (version 2.7.2), launch Denial-of-Service (DoS) flood attacks from an external Kali Linux machine against an internal Ubuntu workstation, and configure firewall rules to stop the attack at the network boundary.

Rather than just setting up basic connectivity, this project focused on how traffic actually behaves across different network segments and how a stateful firewall handles high-volume packet floods. The key technical goals completed during this lab include:

- **Network Segmentation and Routing:** Separated the external attacker network (192.168.71.0/24) from the internal LAN (192.168.1.0/24) using pfSense as a dual-homed gateway. When the local host network conflicted with the 192.168.1.0/24 subnet, I diagnosed the routing leak with traceroute and configured a permanent static route in Kali Linux so traffic consistently passed through the pfSense WAN interface (192.168.71.159).
- **DoS Attack Simulation and Packet Analysis:** Used hping3 on Kali Linux to generate high-speed ICMP Echo floods and TCP Port 80 SYN floods against the Ubuntu target (192.168.1.100). I captured the live traffic in Wireshark on the Ubuntu machine and monitored the pfSense state table as active connections climbed to 57,197 states (14.57 MiB).
- **Perimeter Firewall Mitigation:** Created a top-priority WAN Block rule in pfSense to drop malicious traffic from the attacker IP before it could enter the LAN. Re-testing the attack confirmed that pfSense blocked 155.64 MiB of TCP flood traffic at the WAN interface while zero attack packets reached the Ubuntu machine.

2. Network Topology and Lab Environment

The lab environment runs on three virtual machines inside VMware Workstation: Kali Linux (acting as the external attacker), pfSense CE 2.7.2 (acting as the perimeter firewall and LAN router), and Ubuntu Desktop (acting as the protected internal client). Because my physical home network already uses the 192.168.1.0/24 subnet, bridging the WAN adapter directly to my Wi-Fi caused an IP address conflict with the pfSense LAN. To solve this cleanly, I placed the Kali machine and the pfSense WAN interface on VMware's NAT network (192.168.71.0/24) to represent the outside network, and connected the pfSense LAN interface and the Ubuntu machine to an isolated internal LAN segment.

2.1 Lab Machine Specifications

Virtual Machine	Role	Interface	IP Address / Subnet	Gateway & Network Details
Kali Linux	External Attacker	eth0	192.168.71.160/24	VMware NAT (WAN). Static route for 192.168.1.0/24 via 192.168.71.159.
pfSense CE 2.7.2	Perimeter Firewall	em0 (WAN) em1 (LAN)	192.168.71.159/24 192.168.1.1/24	WAN on VMware NAT; LAN on isolated internal segment with DHCP enabled.

Ubuntu Desktop	Internal Target	ens33	192.168.1.100/24	Isolated internal LAN segment. Default gateway set to 192.168.1.1.

2.2 End-to-End Traffic Flow

- **Stage 1 (Baseline Routing):** When Kali Linux (192.168.71.160) sends packets to Ubuntu (192.168.1.100), Kali's static route directs the packets to the pfSense WAN interface (192.168.71.159). pfSense checks its WAN firewall rules from top to bottom. If an Allow rule matches, pfSense creates a state entry and forwards the packet out its LAN interface (192.168.1.1) to Ubuntu.
- **Stage 2 (Unmitigated DoS Flood):** Using hping3 in flood mode, Kali sends thousands of ICMP Echo and TCP SYN packets per second. Because the initial WAN Pass rule allows traffic from 192.168.71.160, every flood packet passes through the firewall, filling up the pfSense state table and overwhelming Ubuntu's ens33 network interface.
- **Stage 3 (Firewall Mitigation):** A Block rule targeting source IP 192.168.71.160 and destination IP 192.168.1.100 is placed at the very top of the pfSense WAN rule list. Because pfSense evaluates rules on a first-match basis, all incoming flood packets are dropped immediately at the WAN boundary before reaching the internal LAN.

3. Network Configuration and Routing Troubleshooting

3.1 Attacker Network Setup (Kali Linux)

First, I configured the Kali Linux machine on the external NAT segment and checked its interface assignment using the ip a command. As shown in Figure 1, the eth0 adapter received the IP address 192.168.71.160/24.

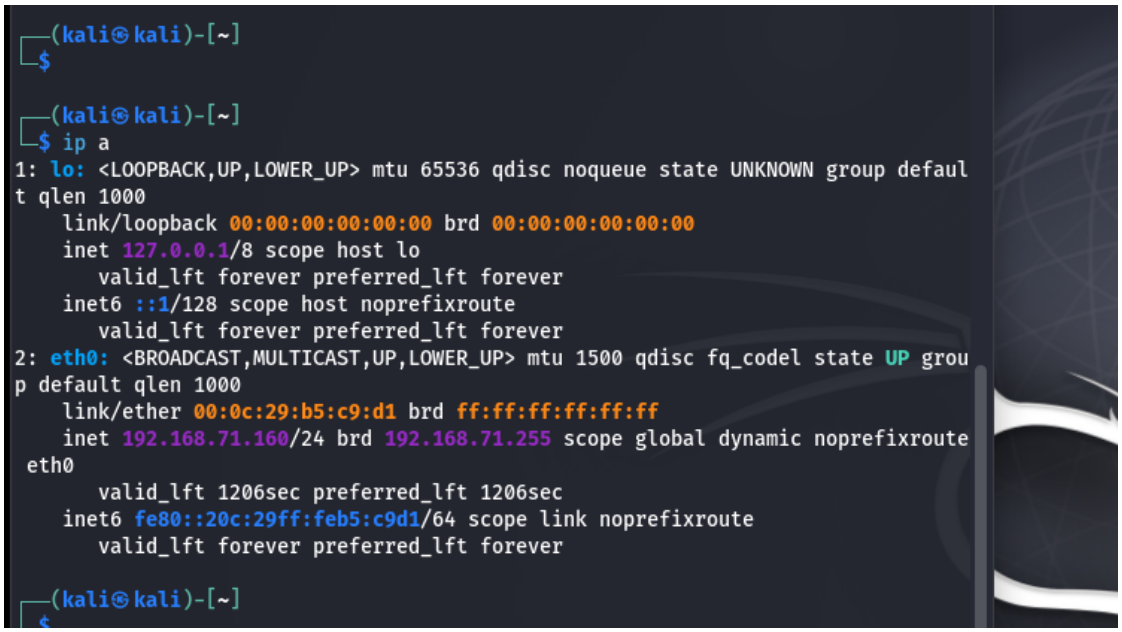


Figure 1: Kali Linux terminal (ip a) showing interface eth0 assigned 192.168.71.160/24.

3.2 Diagnosing and Fixing the Subnet Routing Conflict

When I first tried to ping and run attacks from Kali to the Ubuntu target (192.168.1.100), the pfSense firewall showed zero incoming packets (0/0 B) on its WAN rules. To find out where the packets were going, I ran a traceroute from Kali to 192.168.1.100. As shown in Figure 2, Kali was sending the traffic to its default VMware NAT gateway (192.168.71.2) instead of the pfSense WAN address (192.168.71.159).

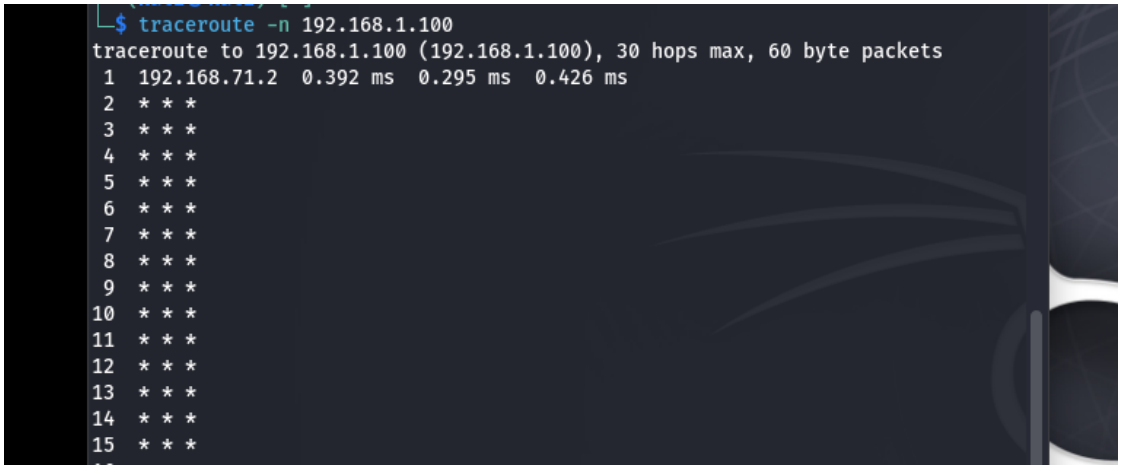


Figure 2: Initial traceroute from Kali showing packets going to the default NAT gateway (192.168.71.2) instead of pfSense.

To fix this, I added a static route in Kali telling the system to send all traffic for the 192.168.1.0/24 network directly through the pfSense WAN IP (192.168.71.159):

sudo ip route replace 192.168.1.0/24 via 192.168.71.159 dev eth0

Because routes added with the ip route command are erased whenever the virtual machine shuts down, I also added this route permanently inside Kali's NetworkManager IPv4 settings. After flushing the route cache, I ran traceroute and ping again. Figure 3 confirms that Hop 1 now hits pfSense (192.168.71.159), Hop 2 reaches Ubuntu (192.168.1.100), and pings complete with 0% packet loss.

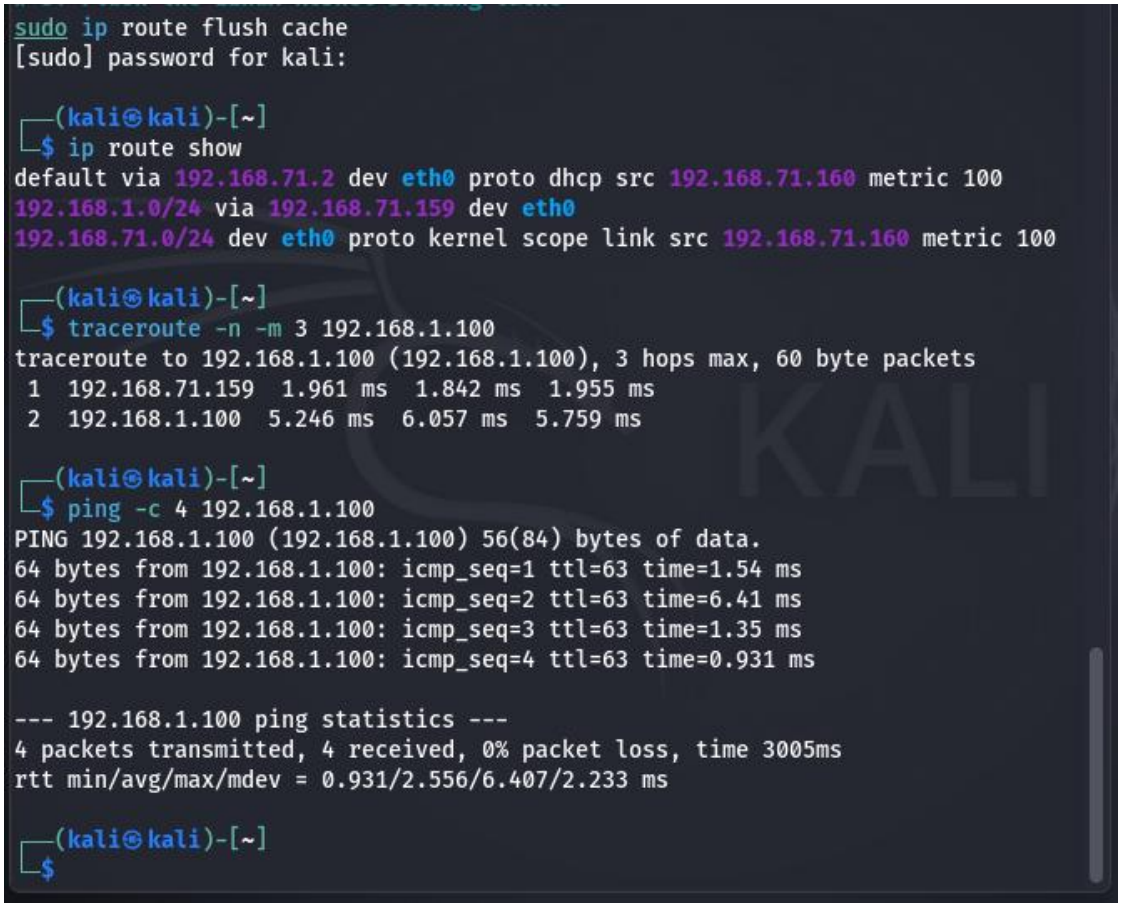


Figure 3: Verified routing table, 2-hop traceroute through pfSense (192.168.71.159) to Ubuntu (192.168.1.100), and 0% ping packet loss.

3.3 LAN DHCP Server and Outbound NAT Verification

On the internal network, pfSense was configured with a LAN IP of 192.168.1.1/24 and an active DHCP server to automatically assign IP addresses to internal machines. Ubuntu received the address 192.168.1.100. Before running the attack, I tested outbound internet access from Ubuntu by pinging Google's public DNS server (8.8.8.8) to verify that pfSense Outbound NAT was working properly (Figure 4).

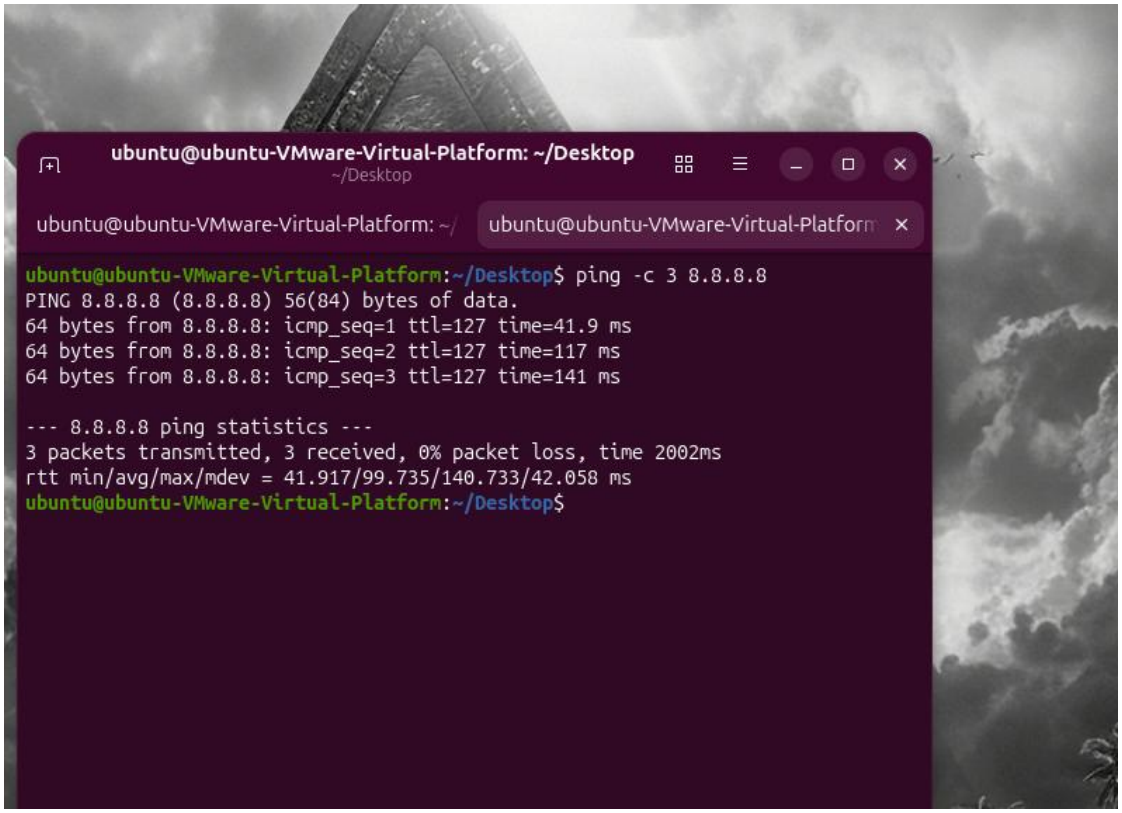


Figure 4: Ubuntu terminal confirming outbound internet connectivity (ping -c 3 8.8.8.8) through pfSense NAT.

Next, I installed Wireshark on Ubuntu and launched it with root privileges (sudo wireshark) so it could capture live traffic on the ens33 network interface. Figure 5 shows the pfSense LAN DHCP configuration alongside Wireshark capturing normal ICMP Echo request and reply packets between Kali (192.168.71.160) and Ubuntu (192.168.1.100).

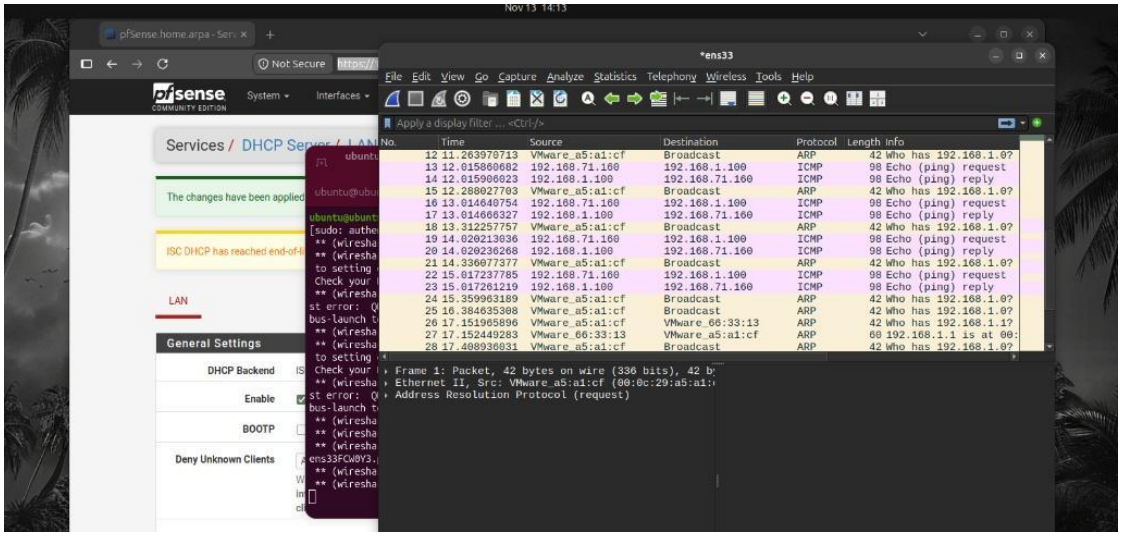


Figure 5: pfSense LAN DHCP Server settings and Wireshark on ens33 capturing baseline ICMP ping traffic.

4. DoS Attack Simulation and Traffic Analysis

With baseline connectivity verified and an initial WAN Pass rule allowing traffic from Kali to the Ubuntu subnet, I used hping3 on Kali Linux to simulate two types of Denial-of-Service floods: an ICMP Echo flood and a TCP Port 80 SYN flood.

sudo hping3 -1 --flood 192.168.1.100

sudo hping3 -S -p 80 --flood 192.168.1.100

The --flood flag instructs hping3 to send packets as fast as the processor allows without waiting for replies. Switching to the Ubuntu VM immediately showed the impact of the attack. As seen in Figure 6, Wireshark captured over 142,680 ICMP packets within seconds, forcing Ubuntu to constantly process incoming requests and generate replies.

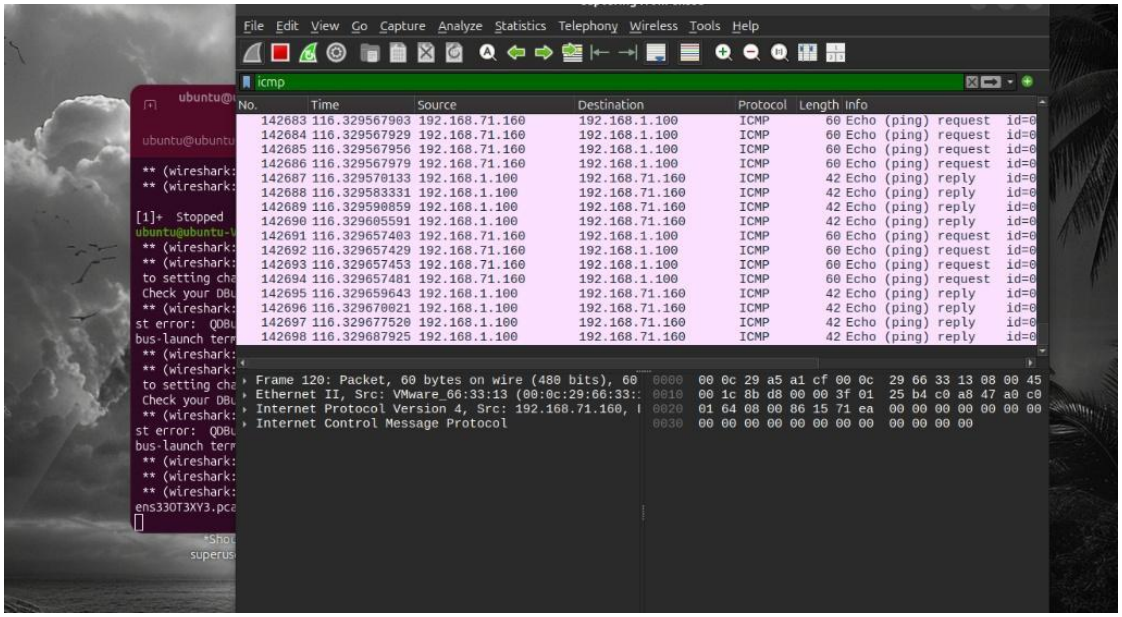


Figure 6: Wireshark on Ubuntu (ens33) flooded with over 142,680 ICMP Echo requests from Kali (192.168.71.160).

I then checked the firewall's internal state table under Diagnostics > States in the pfSense WebGUI during the TCP SYN flood. As shown in Figure 7, every single spoofed or rapid connection attempt on port 80 created a separate state entry (192.168.71.160 -> 192.168.1.100:80) stuck in TIME_WAIT. In a production environment, this rapidly exhausts firewall memory and prevents legitimate users from opening new connections.

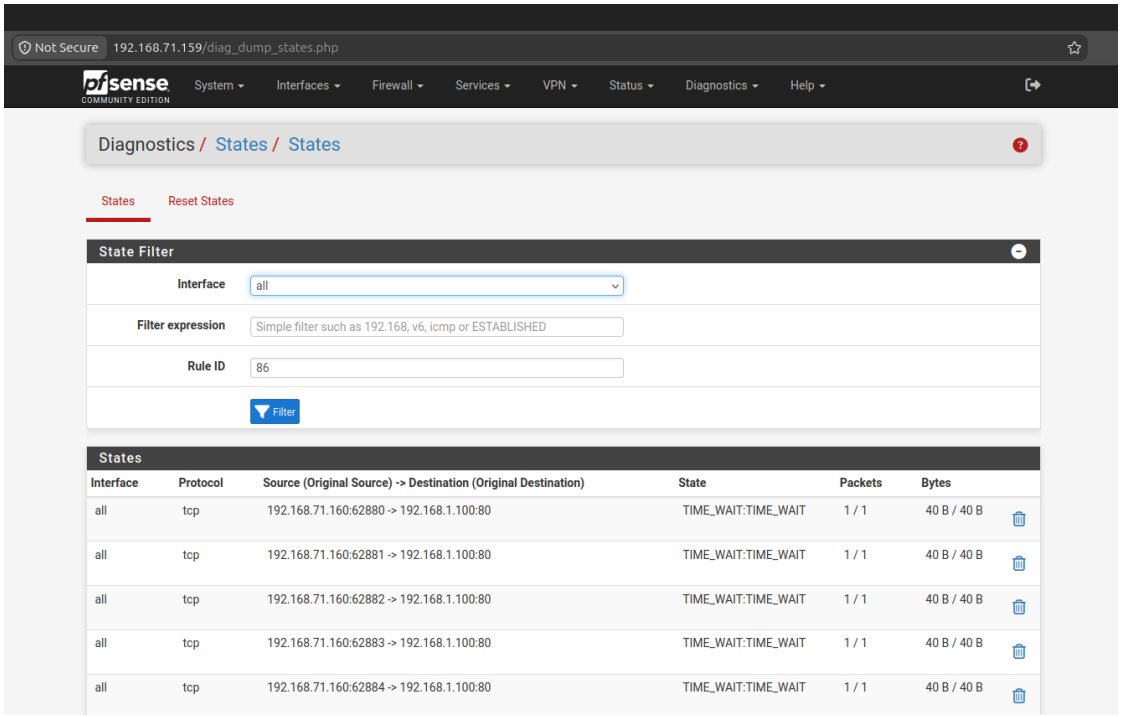


Figure 7: pfSense Diagnostics > States table filled with concurrent TCP port 80 flood connections from 192.168.71.160.

Looking at the Firewall > Rules > WAN screen before adding the block rule (Figure 8), the States counter on the Allow rule reached 57.197K active states and 14.57 MiB of flood traffic passing straight into the LAN.



Figure 8: pfSense WAN Allow rule showing 57.197K states and 14.57 MiB of unmitigated flood traffic.

5. Firewall Rule Mitigation and Verification

To stop the attack at the firewall perimeter, I created a new rule under Firewall > Rules > WAN in pfSense. Two important design choices were made when configuring this rule:

- **Top-Down Rule Ordering (Row 1):** pfSense reads firewall rules from top to bottom and applies the first rule that matches the packet. I used the 'Add to Top' button so the new Block rule sat at Row 1 directly above the existing Allow rule. If the Block rule had been placed below the Allow rule, pfSense would have matched the Allow rule first and let the attack continue.
- **Using Block Instead of Reject:** When pfSense is set to Reject, it sends a TCP Reset (RST) or ICMP Unreachable packet back to the sender for every dropped packet. During a high-speed flood, sending thousands of reset packets wastes firewall CPU and clogs upstream bandwidth. Choosing Block tells pfSense to silently drop the attack packets at the WAN interface.

After saving and applying the Block rule (Action: Block, Interface: WAN, Protocol: IPv4 TCP, Source: 192.168.71.160, Destination: 192.168.1.100), I started a fresh capture in Wireshark on Ubuntu and ran the hping3 TCP SYN flood again from Kali Linux.

Figure 9 shows the final result side by side. On the left, the pfSense WAN Block rule intercepted and dropped 155.64 MiB of TCP flood traffic while the Allow rule underneath stayed at 0/0 B. On the right, Wireshark on Ubuntu's ens33 interface recorded zero packets from 192.168.71.160—only normal background WebGUI (port 443), ARP, and NTP clock sync packets appeared.

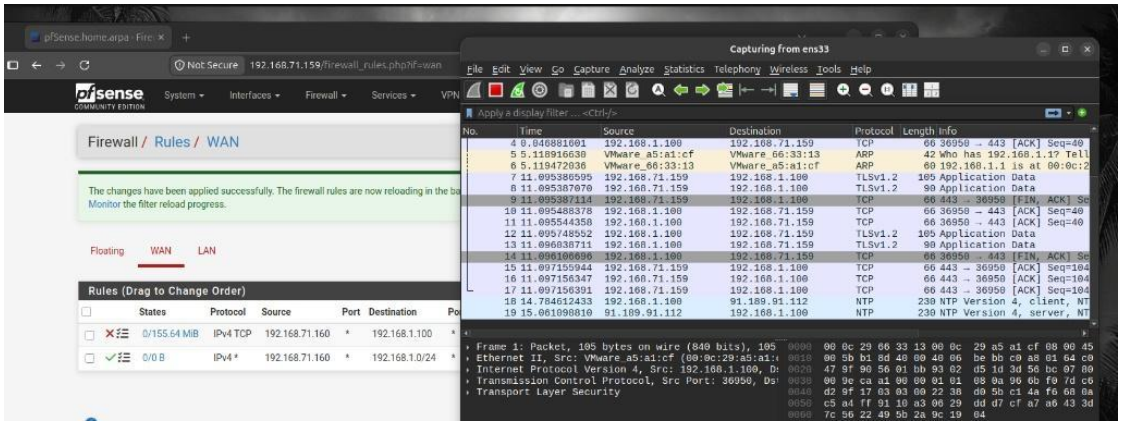


Figure 9: Final verification showing pfSense blocking 155.64 MiB of TCP flood traffic on WAN while Wireshark on Ubuntu shows zero packets from 192.168.71.160.

6. Conclusion and Key Takeaways

Completing this lab demonstrated how network routing, packet analysis, and stateful firewall rules work together in practice. Troubleshooting the initial VMware NAT routing conflict showed why checking the actual packet path with traceroute and ip route is essential before blaming firewall rules. Finally, observing the attack in both Wireshark and the pfSense state table made it clear how quickly an unmitigated SYN or ICMP flood can overwhelm network resources—and how a properly ordered perimeter Block rule can completely isolate and protect the internal LAN.